ISLAMIC UNIVERSITY OF TECHNOLOGY

SWE 4301

LECTURE 8

Topic: Single Responsibility Principle

December 5, 2025

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1 Cohesion **3**

2 SRP: Single Responsibility Principle **4**

2.1 What Is a Responsibility? 5

2.2 Main reason to follow SRP 7

1 Cohesion

Definition, In computer programming, cohesion refers to the degree to which the elements inside a module belong together. In one sense, it is a measure of the strength of relationship between the methods and data of a class and some unifying purpose or concept served by that class. In another sense, it is a measure of the strength of relationship between the class's method and data themselves.
we have the other definition:

Cohesion represents the clarity of the responsibilities of a module.

Cohesion represents the clarity of the responsibilities of a module.
So, cohesion focuses on how single module/class is designed. Higher the cohensiveness of the module/class, better is the OO design.
If our module performs one task and nothing else or has a clear purpose, our module has high cohesion. On the other hand, if our module tries to encapsulate more than one purpose or has an unclear purpose, our module has low cohesion. Cohesion is often contrasted with coupling. High cohesion often correlates with loose coupling, and vice versa. Cohesion is increased if:

- The functionalities embedded in a class, accessed through its methods, have much in common.
- Methods carry out a small number of related activities, by avoiding coarsely grained or unrelated sets of data.

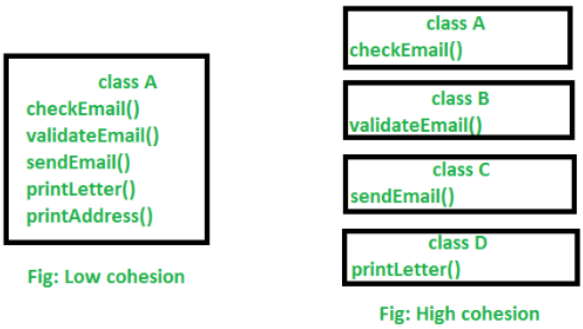

Figure 1: Cohesion example

Advantages of high cohesion,

- Reduced module complexity (they are simpler, having fewer operations).

- Increased system maintainability, because logical changes in the domain affect fewer modules, and because changes in one module require fewer changes in other modules.
- Increased module reusability, because application developers will find the component they need more easily among the cohesive set of operations provided by the module.

2 SRP: SINGLE RESPONSIBILITY PRINCIPLE

They called it cohesion. They defined cohesion as the functional relatedness of the elements of a module.

A class should have only one reason to change.

Or we can extend and say Every thing you write – every module, class, interface, or method, should have one job. It should do the whole job and only that job. Similarly, it will have only one reason to change.

Why was it important to separate these two responsibilities into separate classes? Because each responsibility is an axis of change. When the requirements change, that change will be manifest through a change in responsibility amongst the classes. If a class assumes more than one responsibility, then there will be more than one reason for it to change.

If a class has more than one responsibility, then the responsibilities become coupled. Changes to one responsibility may impair or inhibit the ability of the class to meet the others. This kind of coupling leads to fragile designs that break in unexpected ways when changed.

Everything in the class should be related to that single purpose, i.e., be cohesive. It does not mean that your classes should only contain one method or property.
Example,
The Rectangle class has two methods shown. One draws the rectangle on the screen, the other computes the area of the rectangle. Two different applications use the Rectangle class. One application does computational geometry. It uses Rectangle to help it with the mathematics of geometric shapes. It never draws the rectangle on the screen. The other application is graphical in nature. It may also do some computational geometry, but it definitely draws the rectangle on the screen.

This design violates the Single-Responsibility Principle (SRP). The Rectangle class has two responsibilities. The first responsibility is to provide a mathematical model of the geometry of

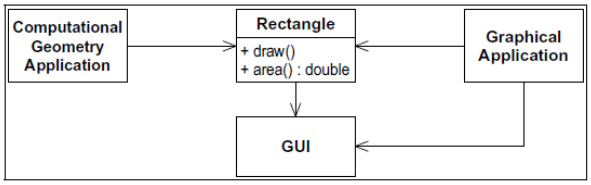

Figure 2: SRP not followed

a rectangle. The second responsibility is to render the rectangle on a graphical user interface.

A better design is to separate the two responsibilities into two completely different classes. This design moves the computational portions of Rectangle into the `GeometricRectangle` class. Now changes made to the way rectangles are rendered cannot affect the `Computational-GeometryApplication`.

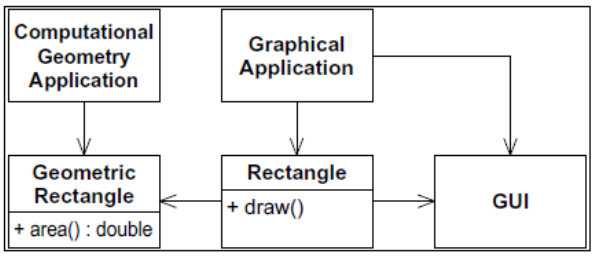

Figure 3: SRP followed for the example

2.1 What Is a Responsibility?

In the context of the SRP, we define a responsibility to be "a reason for change." If you can think of more than one motive for changing a class, then that class has more than one responsibility. This is sometimes hard to see. We are accustomed to thinking of responsibility in groups.

```
//Violation SRP
interface Modem
{
public void dial(String pno);
public void hangup();
public void send(char c);
public char recv();
}
```

Listing 1: `Modem` interface example

Most of us will agree that this interface looks perfectly reasonable. The four functions it declares are certainly functions belonging to a `modem`.

However, there are two responsibilities being shown here. The first responsibility is `connection management`. The second is `data communication`. The `dial` and `hangup` functions manage the connection of the `modem`, while the `send` and `recv` functions communicate data.

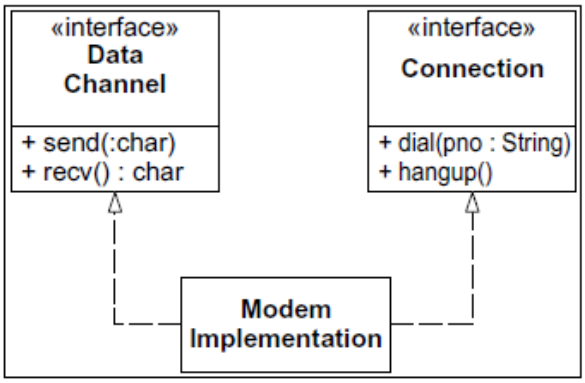

Figure 4: Class diagram modem class following SRP

Should these two responsibilities be separated? That depends on how the application is changing. If the application changes in ways that affect the signature of the connection functions, then the design will smell of Rigidity because the classes that call `send` and `recv` will have to be recompiled and redeployed more often than we like.

If, on the other hand, the application is not changing in ways that cause the the two responsibilities to change at different times, then there is no need to separate them. Indeed, separating them would smell of Needless Complexity.

There is a corollary here. An axis of change is an axis of change only if the changes actually occur. It is not wise to apply the SRP, or any other principle for that matter, if there is no symptom.

Example,

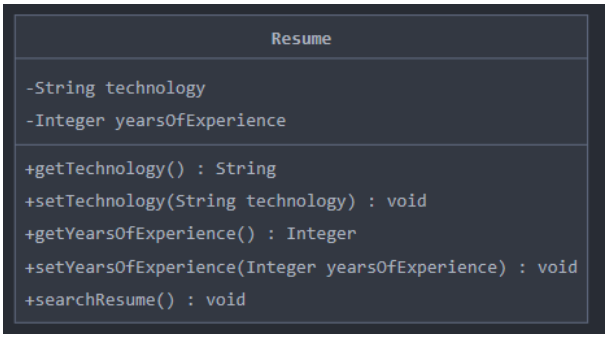

Figure 5: Class diagram for `Resume` without following SRP

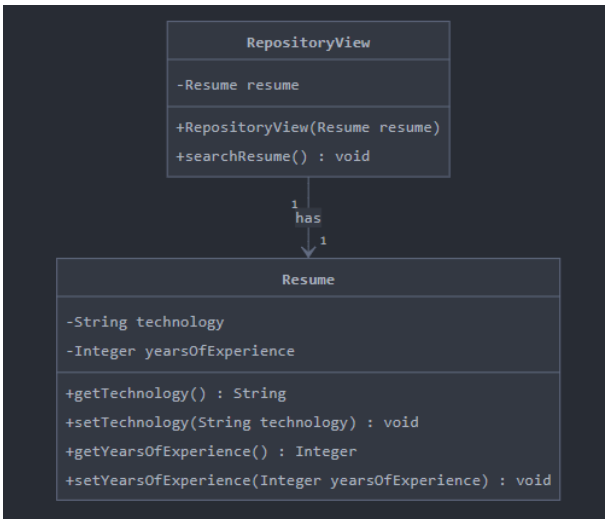

Figure 6: Class diagram for `Resume` following SRP

2.2 Main reason to follow SRP

The number of advantages for using this SOLID principle, as I said before, are many. The following reasons are the most important reasons in terms of impact on development and time:

- **Maintains a manageable project**: Related to the previous advantage. When a class has only one responsibility, your developer brain will only have to think about that one responsibility and no more. This allows you to have a “cleaner” brain of information and allows you to think and manage the project or application better.

- **Enhances maintenance and scalability**: If we think for a second about a class that violates the SRP, we will think about a class that does many things. If for any reason that class has to be modified, it will most likely return errors that did not happen before. If the class does only one thing, you will only have to see to it that you modify that one thing and check that it works well. You don't have to worry that changing one thing will break another.

- **Makes testability better**: As any good developer we know that writing and running unit tests is always advised and considered good practice. By respecting SRP we will find that writing unit tests in the class is an easy task since the class only does one thing and not several and therefore, reduces bug fixes and reduces testing time.

- **Reduces coupling**: If methods only do one thing, one of the things we developers will avoid is coupling. That is, a method will not depend on another method and this makes them more independent of each other.

- **Facilitates code extension**: If the Single Responsibility Principle is met, we have the possibility to combine these classes and build a more user-friendly and modular code. This can be achieved thanks to the dependency injection getting a more testable and extensible code.

To apply the Single Responsibility Principle, a developer should identify the responsibilities of a class and ensure that each responsibility is separated into its own class. This can be achieved by using the following steps:

- Identify the responsibilities of the class: Before applying the Single Responsibility Principle, it is important to understand what the class is responsible for. This can be done by analyzing the methods and properties of the class and determining what they are used for.

- Separate the responsibilities into their own classes: Once the responsibilities of the class have been identified, they can be separated into their own classes. This can be done by creating new classes that are responsible for each of the identified responsibilities. The original class can then delegate its responsibilities to the new classes.

- Ensure that each class has only one responsibility: To ensure that each class has only one responsibility, it is important to review the methods and properties of each class and ensure that they are related to the same responsibility. If a class has multiple responsibilities, it may be necessary to split it into multiple classes.

Example,

```java
public class Car {
    private double fuelLevel;

    public Car(double fuelLevel) {
        this.fuelLevel = fuelLevel;
    }

    public void drive(double distance) {
        if (this.fuelLevel >= distance) {
            this.fuelLevel -= distance;
            System.out.printf("Driving %.2f miles. Fuel level: %.2f%n", distance, this.fuelLevel);
        } else {
            System.out.println("Not enough fuel");
        }
    }

    public void fillGasTank(double amount) {
        this.fuelLevel += amount;
        System.out.printf("Adding %.2f gallons of gas. Fuel level: %.2f%n", amount, this.fuelLevel);
    }
}
```

Listing 2: `CAR` class in Java

To apply the Single Responsibility Principle, we can separate the responsibilities into their own classes. We can create a new class called `FuelTank` that is responsible for maintaining the fuel level.

```java
public class Car {
    private FuelTank fuelTank;

    public Car(FuelTank fuelTank) {
        this.fuelTank = fuelTank;
    }

    public void drive(double distance) {
        if (this.fuelTank.hasEnoughFuel(distance)) {
            this.fuelTank.consumeFuel(distance);
            System.out.printf("Driving %.2f miles. Fuel level: %.2f%n", distance, this.fuelTank.getFuelLevel());
```

```
        } else {
            System.out.println("Not enough fuel");
        }
    }
}

public class FuelTank {
    private double fuelLevel;

    public FuelTank(double fuelLevel) {
        this.fuelLevel = fuelLevel;
    }

    public boolean hasEnoughFuel(double distance) {
        return this.fuelLevel >= distance;
    }

    public void consumeFuel(double distance) {
        this.fuelLevel -= distance;
    }

    public double getFuelLevel() {
        return this.fuelLevel;
    }
}
```

Listing 3: `CAR` and `FUELTANK` class in Java

In this updated example, the `Car` class now has only one responsibility: driving the car. The `FuelTank` class is responsible for maintaining the fuel level.

By separating the responsibilities into their own classes, we have improved the cohesion of the code. The Car class is now focused on driving the car, while the `FuelTank` class is focused on maintaining the fuel level.

Applying the Single Responsibility Principle can also make the code more testable. For example, we can now easily write tests for the `FuelTank` class to ensure that it is correctly maintaining the fuel level.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class FuelTankTest {
    private FuelTank fuelTank;
    @Test
```

```java
    void testInitialFuelLevel() {
        fuelTank = new FuelTank(50.0);
        assertEquals(50.0, fuelTank.getFuelLevel(), 0.001);
    }
    @Test
    void testHasEnoughFuelWhenSufficient() {
        fuelTank = new FuelTank(50.0);
        assertTrue(fuelTank.hasEnoughFuel(30.0));
    }
    @Test
    void testHasEnoughFuelWhenInsufficient() {
        fuelTank = new FuelTank(50.0);
        assertFalse(fuelTank.hasEnoughFuel(70.0));
    }
    @Test
    void testConsumeFuelMultipleTimes() {
        fuelTank = new FuelTank(50.0);
        fuelTank.consumeFuel(20.0);
        fuelTank.consumeFuel(15.0);
        assertEquals(15.0, fuelTank.getFuelLevel(), 0.001);
    }

    @Test
    void testConsumeFuelWhenEmpty() {
        fuelTank.consumeFuel(50.0);
        fuelTank.consumeFuel(10.0);
        assertEquals(-10.0, fuelTank.getFuelLevel(), 0.001);
    }
}
```

Listing 4: `FUELTANK` class test in Java